National Taiwan University
Graduate Institute of Electronics Engineering
Department of Electrical Engineering
Physical Design for Nanometer ICs, Spring 2026

April 20, 2026
Handout #12
Yao-Wen Chang

Programming Assignment #3: Global Placement (due 5pm, Sunday, May 17, 2026; online submission)

ATTENTION: All programming submissions will be checked for duplication against our database, which contains submissions from both previous and current PD classes. Submissions with 40% or higher similarity will be penalized. Most—if not all—students in earlier classes who were flagged for this problem eventually failed the course.

Submission URL & Online Resources:

https://cool.ntu.edu.tw/courses/59874/assignments/386448

1. Problem Statement

In this programming assignment, you are asked to implement a global placer that assigns cells to appropriate positions on a chip. Given a set of modules, a set of nets, and a set of pins for each module, the global placer places all modules within a rectangular chip region.

During global placement, overlaps between modules are allowed. However, modules should be distributed properly so that they can be legalized, that is, placed without overlaps, in the later stages. As illustrated in Figure 1, if modules are not distributed appropriately in global placement (Figure 1(a)), they may still overlap after legalization and detailed placement (see the middle bin in Figure 1(b)). In contrast, Figure 1(c) shows a more appropriate global placement result, which can be legalized successfully, as shown in Figure 1(d).

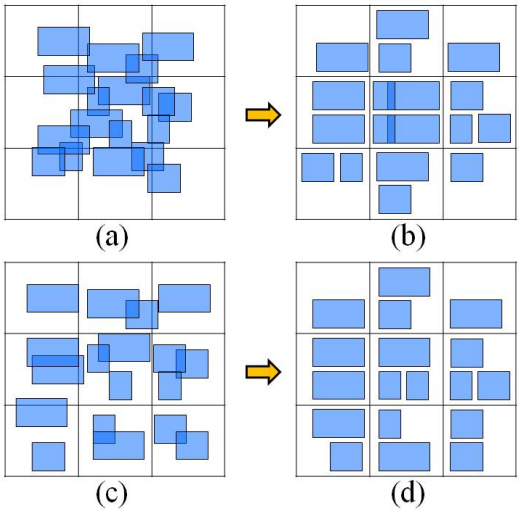

Figure 1. Global placement effects.

In addition to placing modules appropriately, the objective of global placement is to minimize the total net wirelength. The total wirelength W of a set of N is computed by

$$W = \sum_{n_i \in N} HPWL(n_i)$$

where n_i denotes a net in N, and $HPWL(n_i)$ denotes the half-parameter wirelength of n_i. Note that a global placement result that cannot be legalized is unacceptable. In addition, any module placed outside the chip boundary will cause the result to fail.

2. Required Data Structure

```
class Module
{
  public:
  /*get functions*/
    string name();
    double x(); // coordinate of the bottom-left corner
    double y();
    double centerX();
    double centerY();
    double width();
    double height();
    double area();
    unsigned numPins();
    Pin& pin(unsigned index); // index: 0 ~ numPins()-1
  /*set functions*/
    // use this function to set the module position
    Void setPosition(double x, double y);
```

```
};
class Net
{
  public:
    unsigned numPins();
    Pin& pin(unsigned index); // index: 0 ~ numPins()-1
};
class Pin
{
  public:
    double x();
    double y();
    unsigned moduleId();
    unsigned netId();
};
class Placement
{
  public:
    double boundaryTop();
    double boundaryLeft();
    double boundaryBottom();
    double boundaryRight();
    Module& module(unsigned moduleId);
    Net& net(unsigned netId);
    Pin& pin(unsigned pinId);
    unsigned numModules();
    unsigned numNets();
    unsigned numPins();
    double computeHpwl();// compute total wirelength
    // output global placement result file for later stages
    outputBookshelfFormat(string fileName);
};
```

The above functions define the interface of the placement database and can be used to access the data required for global placement. After global placement, you should use the function `setPosition(double x, double y)` to update module coordinates when moving blocks. The coordinates of the pins on each module will then be updated automatically. Note that modules must not be placed outside the chip boundary; otherwise, the program may not execute properly.

3. Compile & Execution

To compile the program, simply type:

```
make
```

Please use the following command line to execute the program:

```
./place -aux <inputFile.aux>
```

For example:

```
./place -aux ibm01-cu85.aux
```

4. Language/Platform

- Language: C/C++
- Platform: Linux. **Please develop your programs on the servers in the EDA Union Lab.**

5. Submission

You need to create a directory named <student id>_pa3/ (for example, f09943000_pa3/) which must contain the following materials:

- A directory named src/ containing your source codes: only *.h, *.hpp, *.c, *.cpp are allowed in src/, and no directories are allowed in src/.
- A directory named bin/ containing your executable binary, named **place**.
- A makefile name makefile or Makefile that produces an executable binary from your source codes by simply typing "make". The binary should be generated under the directory <student id>_pa3/bin/.
- A text readme file named readme.txt describing how to compile and run your program.
- A report named report.pdf describing the data structures used in your program and your findings from this programming assignment.

6. Grading Policy

This programming assignment will be graded based on the following:

(1) correctness of the program,

(2) solution quality,

(3) running time (limited to 2 hours for each case),

(4) report.doc, and

(5) readme.txt.

Please make sure to check all items before submission.

If the global placement result can be legalized, the final solution quality is judged by the total HPWL after the detailed placement stage, which will be displayed on the screen as follows:

```
Benchmark: ibm01

Global HPWL: 2349680    Time:     13.0 sec (0.2 min)
Legal HPWL: 2531684     Time:      1.0 sec (0.0 min)
Detailed HPWL: 2482319     Time:      0.0 sec (0.0 min)
==================================================

       HPWL: 2482319     Time:     14.0 sec (0.2 min)
```

However, if the global placement result cannot be legalized, the solution quality will be evaluated using the following cost function:

$$W \cdot (1 + scaled\_overflow\_per\_bin)$$

where W is the total wirelength obtained from the global placement stage. The "scaled overflow per bin" can be computed by using the following script:

```
perl check_density_target.pl <input.nodes> <Solution PL file> < input.scl>
```

For example:

```
perl check_density_target.pl ibm01.nodes ibm01-cu85.gp.pl ibm01-cu85.scl
```

The "scaled overflow per bin" can then be displayed on the screen as follows:

```
NumRows: 144 are defined
Phase 0: Total 144 rows are processed.
         ImageWindow=(0 0 2295 2304) w/ row_height=16
         Total Row Area=5287680
Phase 1: CMAP Dim: 15 x 15 BinSize: 160 x 160 Total 225 bins.
         NumNodes: 12752 NumTerminals: 246
Phase 2: Node file processing is done. Total 12752 objects (terminal 246)
         Total 12752 entries in ObjectDB
         Total movable area: 4231408
Phase 3: Solution PL file processing is done.
         Total 12752 objects (terminal 246)
Phase 4: Congestion map construction is done.
         Total 12752 objects (terminal 246)
Phase 5: Congestion map analysis is done.
        Total 225 (15 x 15) bins. Target density: 1.000000
         Violation num: 15 (0.066667)    Avg overflow: 0.058200  Max overflow: 0.178813
         Overflow per bin: 39.744101     Total overflow amount: 8942.422742
         Scaled Overflow per bin: 0.018294
```

Note that solutions that can be legalized will receive higher scores than those that cannot.

The final score for each case is first determined by a "temporary score" from our

evaluator (only for legalized placement results). We will then linearly scale the scores based on the top score for each case. Specifically, the top score for each case will be scaled to 10, and all other scores will be scaled proportionally. You can obtain your temporary score from the evaluator using the command below:

```
bash evaluator/evaluator <HPWL> <Time (s)>
```

For example, if you want to run the evaluator for the placement result of your placer, the command is as follows:

```
bash evaluator/evaluator 2482319 14
```

The temporary score is computed using the following formula:

$$\text{case score} = \text{quality score} \times 0.8 + \text{runtime score} \times 0.2,$$

where your quality score and runtime score here are determined by interpolation from the quality and runtime tables based on submissions from the Spring 2025 class (pd25). In those tables, the top score is assigned 9.5, the bottom score is assigned 4, and all other scores are distributed evenly in between.

7. Online Resources

Sample codes, benchmarks, and checkers can be found on the submission website. If you plan to implement an analytical method, the journal paper on NTUplace3 is also provided.

8. Hints

Both combinatorial and analytical methods are acceptable for this programming assignment.. If you choose an analytical method, you may refer to the sample code in the following files:

- `Point.h`
- `ObjectiveFunction.h/.cpp`
- `Optimizer.h/.cpp`
- `GlobalPlacer.cpp`

These files provide a point library and a conjugate-gradient optimizer. In addition, we recommend that you read **Section II (Analytical Placement Model)** and **Section III-A (GP, i.e., Global Placement)** of the NTUplace3 paper, as they may provide useful insight for your implementation.